AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_HWSW_task_6 Version : 1	

HW/SW PROTOTYP AV MULTIPROCESSOR SYSTEM

Darwin Rengifo

2020-06-24

Sammanfattning:
Denna rapport beskriver en förstudie och planering för Multipricessor System projektet med användningen av accelerometer från DE10-Lite kortet, två CPU processor, en shared memory och Qsys-verktyget för att integrerar hårdvaran och mjukvaran interface genom läsningen och skrivningen på Avalon bus och en VGA skärmen för att visa resultatet. Konstruktionen använder CPU(2) för läser koordinaterna från accelerometer och spara de på shared memory, CPU(1) för att läser denna koordinater och flytta sig en rektangel på VGA skärmen. Denna rapport innehåller aktivitets- och tidplan, projektkostnad, kravspecifikations, projektarkitektur och beskrivning av utvecklade komponenter.

Innehåll

1. INLEDNING 2
2. KRAVSPECIFIKATION FRÅN KUND 3
3. PROJEKPLAN 4
4. HÅRDVARAN ARKITEKTUR FÖR INGÅENDE KOMPONENTER. 5
5. KONSTRUKTIONS BESKRIVNING 6
5.1 Systemarkitektur 6
5.2 Delsystem 8
5.2.1 Toppnivån 8
5.2.1.1 Systemarkitektur 8
5.3 I/O pinnar 10
6. VALIDERING KONSTRUKTIONEN 11
6.1 Testprotokoll för validering 11
7. MJUKVARAN ARKITEKTUR 12
7.1 Minneskarta 12
7.2 Drivrutiner 12
7.2.1 Timer-drivrutiner 12
7.2.2 VGA färger 13
7.2.3 VGA-drivrutiner 13
7.2.4 SPI Accelerometer-drivrutiner 14
7.3 C koden 15

1. INLEDNING

Denna rapport beskriver hur man bygger ett ”Multiprocessor System”. Konstruktionen ska vara uppdelad på två processorer med ett dubbelportsminne mellan processorerna som hanterar data. En rektangel ska kunna flytta sig på en bildskärm. Rektangeln ska manövreras med en accelerometer.

2. KRAVSPECIFIKATION FRÅN KUND

Tabell 1. Kravspecifikation från kund

Krav	Beskrivning	Utfört
	Funktion	**Ja/nej**
001	En rektangel ska flytta sig på en bildskärm. Rektangeln ska manövreras med fyra tryckknappar (upp, ner, vänster och höger) eller med t.ex. en accelerometer kopplad till kortet. Konstruktionen ska vara uppdelad på två processorer med ett dubbelportsminne mellan processorerna.	Ja
	Andra krav	
002	**CPU_1:** • Ska ansvara för att rita ut en rektangel via VGAkomponenten till skärmen. I dubbelportsminnet finns koordinater (från CPU_2) som bestämmer rektangelns läge på skärmen. • Ska ansvara för att uppdatera systemet när rektangeln flyttas. Detta innebär att den ska radera den gamla rektangeln och rita in den igen på den nya positionen. (Exempel se figur 1)	Ja
003	**CPU_2:** • Ska ansvara för att skanna tryckknapparna (eller t.ex. en accelerometer) och uppdatera koordinaterna i dubbelportsminnet. • Ska skanna av tangenterna 10 gånger i sekunden och hålla reda på rektangelns läge så att den inte går utanför skärmen. • Ska när det är giltiga koordinater, skriva in dessa i dubbelportsminnet. • Ska skriva ut koordinaterna på PC skärmen (printf).	Ja
004	Regler och riktlinjer för VHDL och C ska följas.	Ja
005	Rapporten ska också innehålla kostnaden för konsultuppdraget. Företaget konstruktören är anställd på fakturerar 600 SEK/timme för projektet. Den totala kostnaden blir antal timmar * 600 SEK. Frivilligt tillägg är att redovisa till kunden med en kort Youtube film som beskriver projektet.	Ja
	Leverans	
006	Leveransen ska ske till plattformen Itslearning. Leveransen ska vara en kort rapport med C-filen, det arkiverade HW-projektet och testprotokollet. Namnet på filen ska vara ”förnamn_efternamn_HWSW_task_6”. Sista leveransdag se kurs schema (för VG).	Ja

3. PROJEKPLAN

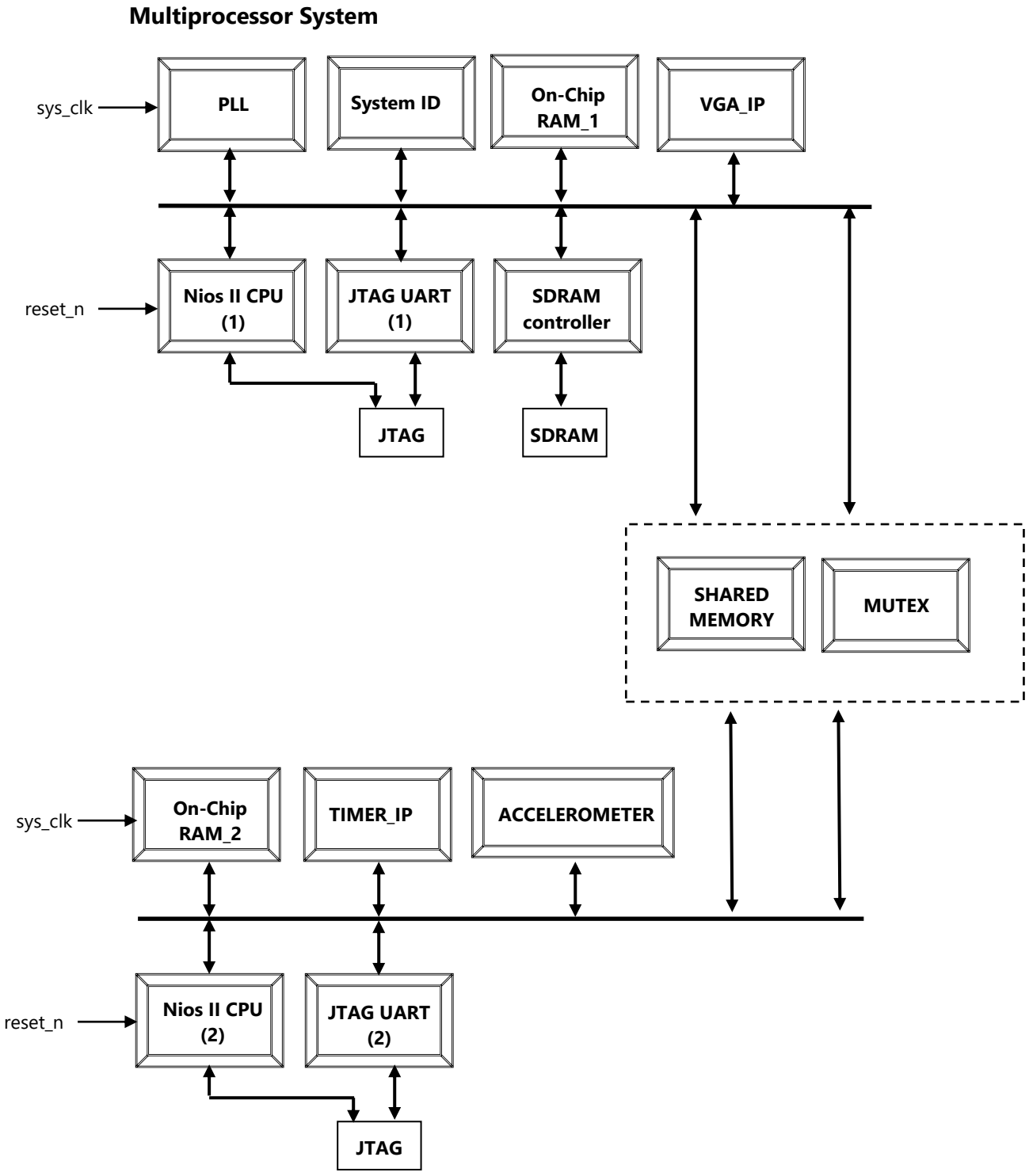
Aktivitets - och tidplan för ingenjörsjobbet kan ses i Tabell 2. Totalt uppskattas ingenjörsjobbet ta 28 timmar/600kr(timme), vilket ger en lönekostnad på 16 800 SEK.

Tabell 2. Aktivitets- och tidsplan för ingenjörsjobbet

Aktivitet	Veckor				
	18	19	20	Timmar(h)	Kostnad(kr)
Kravspec./förstudie	5			5	3 000
Testprotokoll/ Drivrutiner	1	1	1	3	1 800
Arkitekturbeskrivning	3	3		6	3 600
Programmering av visuellt gränssnitt	4	3	1	7	4 200
Verifiering		1	1	2	1 200
Validering			1	1	6 00
Slutrapport		2	2	4	2 400
Summa tid	13	10	6	28	16 800

4. HÅRDVARAN ARKITEKTUR FÖR INGÅENDE KOMPONENTER.

Systemarkitekturen som FPGA:n programmerats med två CPU, shared memory för att delar information, mutex för att blockerar registret i shared memory och en enkel bussarkitektur för att läsa och skriver från IP komponenten med en mikroprocessor av typen Nios II som styr, se Figur 1.



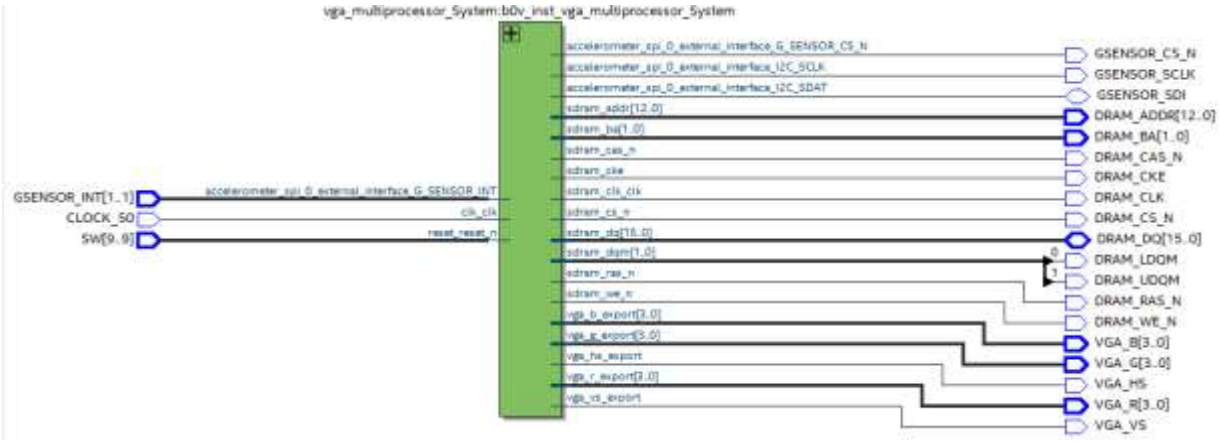
Figur 1. Systemarkitektur, blockschema för MULTIPROCESSOR SYSTEM

5. KONSTRUKTIONS BESKRIVNING

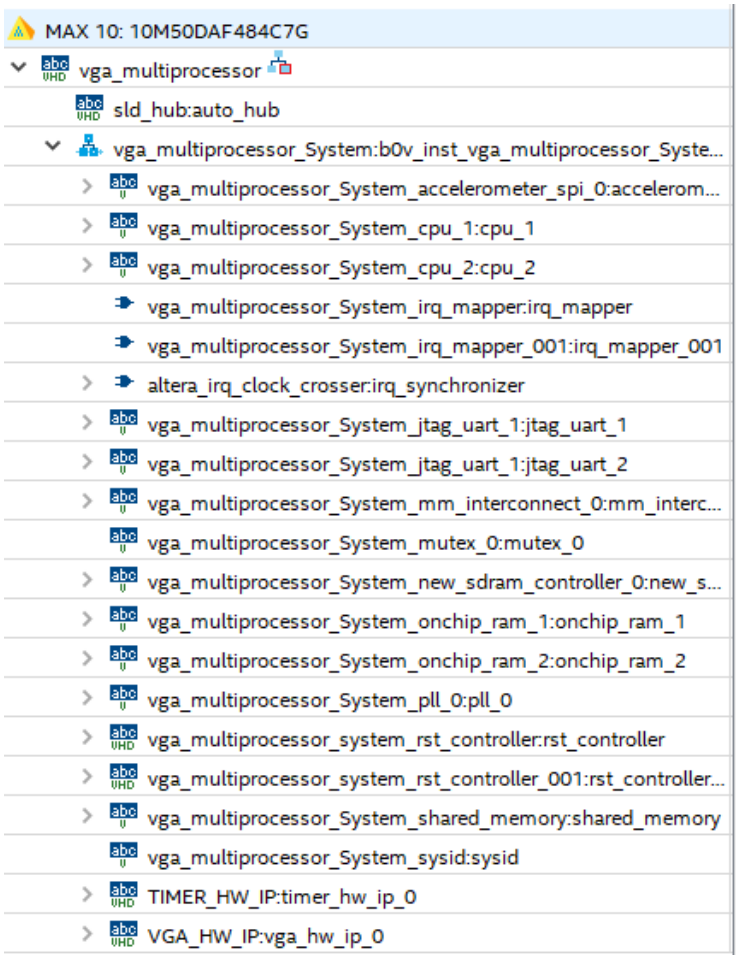
Programmet körs på kortet DE10-Lite är en robust hårdvara design plattform som byggde kring Altera MAX 10 FPGA med en Nios II-CPU som är anpassad för ingenjörsjobbet. DE10-Lite-kortet innehåller en accelerometer som skicka koordinater som kan kontrollera efter behov.

5.1 Systemarkitektur

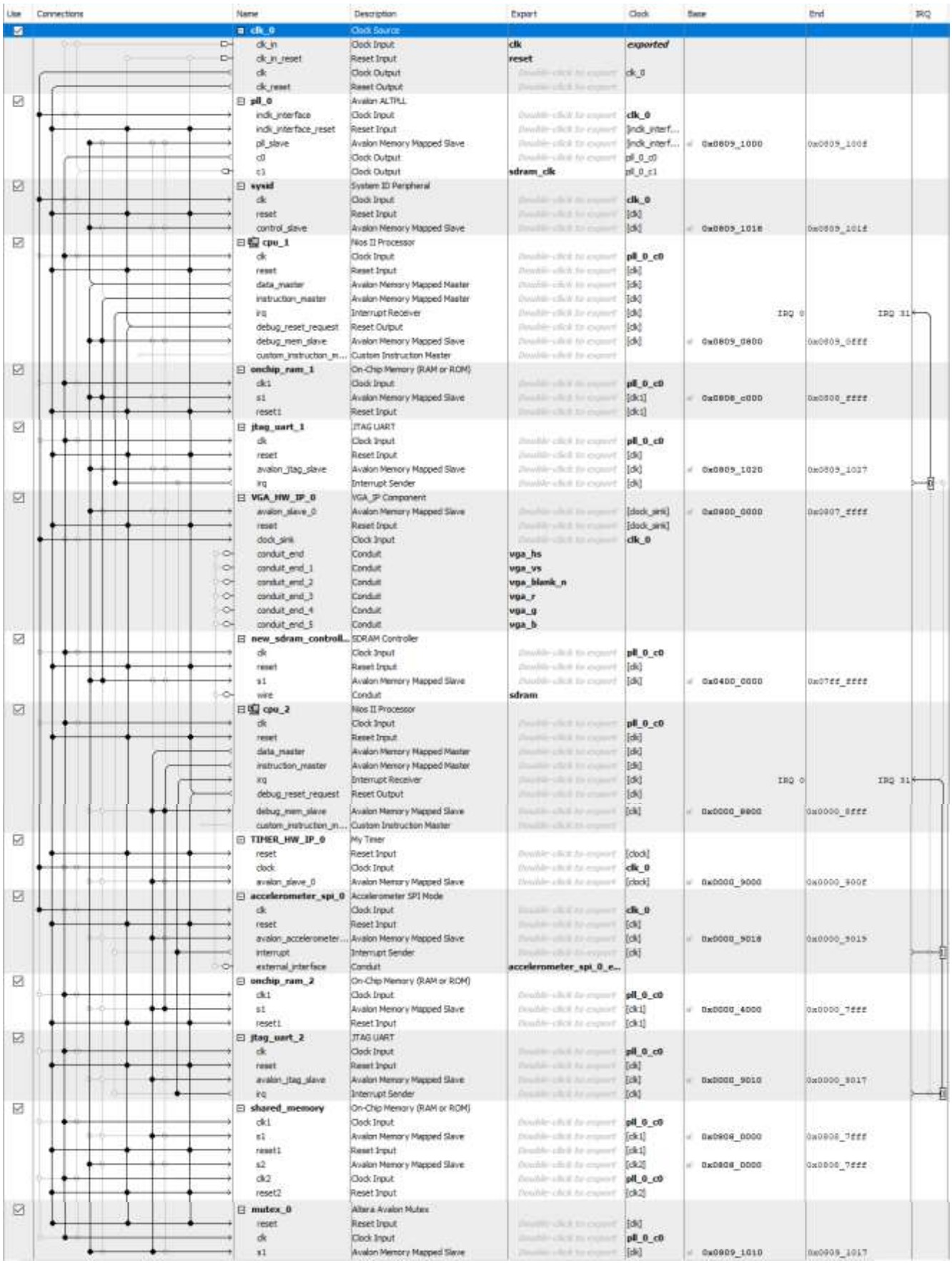
TOPPNIVÅN arkitektur visas i Figur 2, i figur 3 visar komponenthierarki och i figur 4 visar QSYS interface.



Figur 2. Systemarkitekturen för för MULTIPROCESSOR SYSTEM



Figur 3. Komponenthierarki för för MULTIPROCESSOR SYSTEM



Figur 4. QSYS interface för MULTIPROCESSOR SYSTEM

5.2 Delsystem

Systemet innehålla följande komponenter:

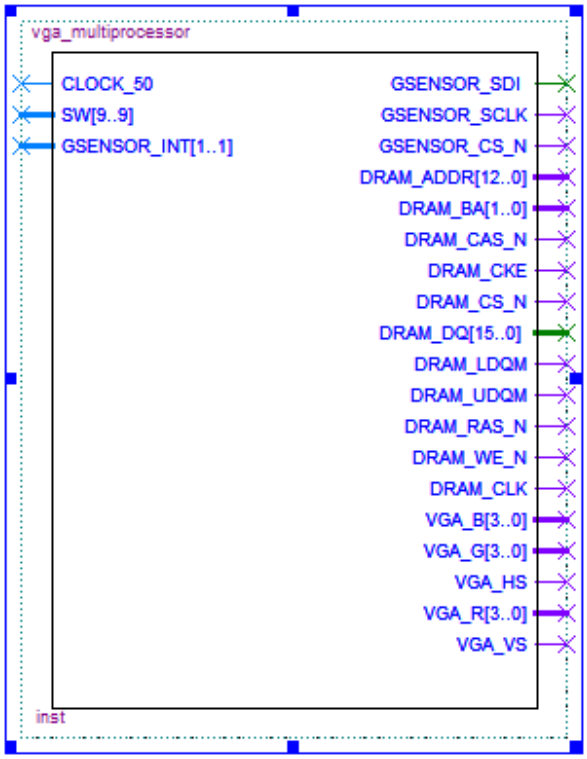
5.2.1 Toppnivån

Komponent namn: ***VGA_MULTIPROCESSOR***

5.2.1.1 Systemarkitektur

Den toppnivån användas för att instansierar Qsys interface son innehålla hela IP komponenterna. Toppnivån får insignaler från accelerometer och skickar utsignaler till VGA skärmen.

TOPPNIVÅN symbol visas i Figur 5 och i Tabell 3 visar In- och utsignalerna.



Figur 5. TOPPNIVÅN för MULTIPROCESSOR SYSTEM

Tabell 3. In- och utsignalerna till TOPP filen

Signal	Namn	Riktning	Typ
Klocksignal 50 MHz	CLOCK_50	in	std_logic
Switch reset	SW	in	std_logic
Interrupt pin	GSENSOR_INT	in	std_logic_vector(1 downto 1)
I2C serial data	GSENSOR_SDI	out	std_logic
I2C serial clock	GSENSOR_SCLK	out	std_logic
Chip Select	GSENSOR_CS_n	out	std_logic
Horizontell synkronism	vga_hs	out	std_logic

Vertikal synkronism	vga_vs	out	std_logic
Röd signal	vga_r	out	std_logic_vector(3 downto 0)
Grön signal	vga_g	out	std_logic_vector(3 downto 0)
Blå signal	vga_b	out	std_logic_vector(3 downto 0)
External minne	SDRAM	out	std_logic, std_logic_vector

Ingången signaler

CLOCK_50: 50Mhz input clock.
sw(reset_n): Reset av signalen. Active low.
gsensor_int: Interrupt pin 1.

Utgången signaler

SDRAM: Externa minne som innehålla data och instruktioner.
vga_hs: Horizontell synkronism. Den kontrollerar början och slutet på en rad och ser till att skärmen visar pixlarna mellan vänster och höger del av den synliga delen av bildskärmen.
vga_vs: Vertikal synkronism. Kontrollerar början och slutet på en ram(framme) och ser till att den visas mellan topp och nedre kanten och inom skärmen synliga ram, alla linjer som sminka.
vga_r: Definierar röd färg av signalen.
vga_g: Definierar grön färg av signalen.
vga_b: Definierar blå färg av signalen.

5.3 I/O pinar

Figur 6 visar pinar för DE10-Lite kortet.

Node Name	Direction	Location
CLOCK_50	Input	PIN_N14
DRAM_ADDR[12]	Output	PIN_R20
DRAM_ADDR[11]	Output	PIN_P20
DRAM_ADDR[10]	Output	PIN_T20
DRAM_ADDR[9]	Output	PIN_P19
DRAM_ADDR[8]	Output	PIN_P18
DRAM_ADDR[7]	Output	PIN_R18
DRAM_ADDR[6]	Output	PIN_T19
DRAM_ADDR[5]	Output	PIN_T18
DRAM_ADDR[4]	Output	PIN_U19
DRAM_ADDR[3]	Output	PIN_U18
DRAM_ADDR[2]	Output	PIN_V18
DRAM_ADDR[1]	Output	PIN_W19
DRAM_ADDR[0]	Output	PIN_U17
DRAM_BA[1]	Output	PIN_T22
DRAM_BA[0]	Output	PIN_T21
DRAM_CAS_N	Output	PIN_U21
DRAM_CKE	Output	PIN_N22
DRAM_CLK	Output	PIN_L14
DRAM_CS_N	Output	PIN_U20
DRAM_DQ[15]	Bidir	PIN_F22
DRAM_DQ[14]	Bidir	PIN_G19
DRAM_DQ[13]	Bidir	PIN_G20
DRAM_DQ[12]	Bidir	PIN_G22
DRAM_DQ[11]	Bidir	PIN_H22
DRAM_DQ[10]	Bidir	PIN_H21
DRAM_DQ[9]	Bidir	PIN_J22
DRAM_DQ[8]	Bidir	PIN_P21
DRAM_DQ[7]	Bidir	PIN_V21

Node Name	Direction	Location
DRAM_DQ[6]	Bidir	PIN_W20
DRAM_DQ[5]	Bidir	PIN_W22
DRAM_DQ[4]	Bidir	PIN_Y22
DRAM_DQ[3]	Bidir	PIN_AA21
DRAM_DQ[2]	Bidir	PIN_AA22
DRAM_DQ[1]	Bidir	PIN_Y20
DRAM_DQ[0]	Bidir	PIN_Y21
DRAM_LDQM	Output	PIN_V22
DRAM_RAS_N	Output	PIN_U22
DRAM_UDQM	Output	PIN_J21
DRAM_WE_N	Output	PIN_V20
GSENSOR_CS_N	Output	PIN_AB16
GSENSOR_INT[1]	Input	PIN_Y14
GSENSOR_SCLK	Output	PIN_AB15
GSENSOR_SDI	Bidir	PIN_V11
SW[9]	Input	PIN_F15
VGA_B[3]	Output	PIN_N2
VGA_B[2]	Output	PIN_P4
VGA_B[1]	Output	PIN_T1
VGA_B[0]	Output	PIN_P1
VGA_G[3]	Output	PIN_R1
VGA_G[2]	Output	PIN_R2
VGA_G[1]	Output	PIN_T2
VGA_G[0]	Output	PIN_W1
VGA_HS	Output	PIN_N3
VGA_R[3]	Output	PIN_Y1
VGA_R[2]	Output	PIN_Y2
VGA_R[1]	Output	PIN_V1
VGA_R[0]	Output	PIN_AA1
VGA_VS	Output	PIN_N1

Figur 6. Pinnar för DE10-Lite kortet

6. VALIDERING KONSTRUKTIONEN

6.1 Testprotokoll för validering

I Tabell 4 visar testprotokoll för validering och länken visar en video for att validerats systemet.

Tabell 4. Testprotokoll för validering

Testfall	Beskrivning	Acceptansbeskrivning	Verifiering
1	Gör en reset(SW(9)) av systemet.	Reset CPU'n	Ja(se video)
2	CPU'n	Jobbar med två CPU's	Ja(se video)
3	Koordinat X	Visa värde på Skarmen	Ja(se video)
4	Koordinat Y	Visa värde på Skarmen	Ja(se video)
5	Koordinat Z	Visa värde på Skarmen	Ja(se video)
6	Rektangel	Flytta sig på skärmen beroende på koordinaterna	Ja(se video)

https://www.youtube.com/watch?v=hdKtJ5Yam4A

7. MJUKVARAN ARKITEKTUR

7.1 Minneskarta

Memory-map från ”sumary.html” för konstruktionen, se figur 10.

Nios II Memory Map

Slave Descriptor	Address Range	Size	Attributes
jtag_uart_1	0x08091020 - 0x08091027	8	printable
sysid	0x08091018 - 0x0809101F	8	
mutex_0	0x08091010 - 0x08091017	8	
pll_0	0x08091000 - 0x0809100F	16	
onchip_ram_1	0x0808C000 - 0x0808FFFF	16384	memory
shared_memory	0x08080000 - 0x08087FFF	32768	memory
VGA_HW_IP_0	0x08000000 - 0x0807FFFF	524288	
new_sdram_controller_0	0x04000000 - 0x07FFFFFF	67108864	memory

Figur 7. Flödesdiagram CPU(1)

Nios II Memory Map

Slave Descriptor	Address Range	Size	Attributes
mutex_0	0x08091010 - 0x08091017	8	
shared_memory	0x08080000 - 0x08087FFF	32768	memory
accelerometer_spi_0	0x00009018 - 0x00009019	2	
jtag_uart_2	0x00009010 - 0x00009017	8	printable
TIMER_HW_IP_0	0x00009000 - 0x0000900F	16	
onchip_ram_2	0x00004000 - 0x00007FFF	16384	memory

Figur 8. Flödesdiagram CPU(2)

7.2 Drivrutiner

Det visar en översikt över de funktioner som krävdes av kravspecifikationen.

7.2.1 Timer-drivrutiner

För att använda timer-drivrutinen finns de macron som är listade i Tabell 5.

Tabell 5. Timmer-drivrutiner i altera_avalon_timer_regs.h

Funktion	Beskrivning
#define TIMER_STOP IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x00000000)	Stoppar timern
#define TIMER_RESET IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x40000000)	Nollställer timern
#define TIMER_START IOWR_32DIRECT(TIMER_HW_IP_0_BASE,4,0x80000000)	Startar timern

#define TIMER_READ IORD_32DIRECT(TIMER_HW_IP_0_BASE,0)	Läser ut timervärdet

7.2.2 VGA färger

Värde (hexadecimalt)	Färg
0x00	
0x01	
0x02	
0x03	
0x04	
0x05	
0x06	
0x07	

7.2.3 VGA-drivrutiner

Tabell 6: "DE10_Lite_VGA_Driver.h" innehåller funktionsdeklarationer och inkluderas för att få tillgång till vga-rutinerna.

Tabell 7: ***"DE10_Lite_VGA_Driver.h"*** innehåller också definitioner av macron som fungerar för att läser och skriver värden i minnesplacering.

Tabell 6. Funktionsdeklarationer i DE10_Lite_VGA_Driver.h

Funktion	Beskrivning
void clear_screen(alt_u8 rgb);	Rensar skärmen med färgen rgb.
void clear_del_screen(alt_u32 x_start, alt_u32 y_start, alt_u32 x_end, alt_u32 y_end, alt_u32 color);	Rensar skärmen med beroende på koordinaterna.
void print_str(alt_u32 x, alt_u32 y, alt_u8 rgb, alt_u8 bg_rgb, alt_u8 *str);	Skriver ut en sträng på koordinaterna x,y med färgen rgb och bakgrundsfärgen bg_rgb. Skulle strängen fortsätta utanför högerkanten fortsätter den på raden under. Skulle den gå utanför nederkanten fortsätter den längst upp till vänster.
void int_print(alt_32 x_start, alt_32 y_start, int data, int data_l, alt_32 color, alt_u32 BGcolor);	Skriver ut siffror(data) med antal siffror(data_l) på koordinaterna x,y med färgen color och bakgrundsfärgen BGrgb.

Tabell 7. VGA-macron i DE10_Lite_VGA_Driver.h

Funktion	Beskrivning
#define write_pixel(x,y,color) IOWR_32DIRECT(**VGA_HW_IP_0_BASE**,((320*(y)+(x))*4),(color))	Skriver värde "color" till position x_pos,y_pos i minnet.
#define read_pixel(x,y) IORD_32DIRECT(**VGA_HW_IP_0_BASE**,(320*(y)+(x))*4)	Läser värde från x_pos, y_pos i minnet.

7.2.4 SPI Accelerometer-drivrutiner

Tabell 8: "altera_up_avalon_accelerometer_spi.h" innehåller funktionsdeklarationer och inkluderas för att få tillgång till accelerometer-rutinerna.

"altera_up_avalon_accelerometer_spi.c" innehåller accelerometer-rutinerna.

"altera_up_avalon_accelerometer_spi_regs.h" innehåller definitioner av macron som fungerar för att läser och skriver värden i minnesplacering.

Tabell 9. Funktionsdeklarationer i altera_up_avalon_accelerometer_spi.h/.c

Funktion	Beskrivning
alt_up_accelerometer_spi_dev* alt_up_accelerometer_spi_open_dev(const char* name);	Öppnar "Accelerometer SPI Mode" enheten som anges av "namn"
int alt_up_accelerometer_spi_read_address_register(alt_up_accelerometer_spi_dev *accel_spi, alt_u8 *addr);	Läser konfigurationsdata från en av inbyggda videoenhet register.
int alt_up_accelerometer_spi_read(alt_up_accelerometer_spi_dev *accel_spi, alt_u8 addr, alt_u8 *data);	Läser data från accelerometer register.
int alt_up_accelerometer_spi_write(alt_up_accelerometer_spi_dev *accel_spi, alt_u8 addr, alt_u8 data);	Skriver data till accelerometer register.
int alt_up_accelerometer_spi_read_x_axis (alt_up_accelerometer_spi_dev *accel_spi, alt_32 *x_axis);	Läser X-axelvärdet från båda registren från accelerometer och omvandlar värdet till ett signed heltal.
int alt_up_accelerometer_spi_read_y_axis (alt_up_accelerometer_spi_dev *accel_spi, alt_32 *y_axis);	Läser Y-axelvärdet från båda registren från accelerometer och omvandlar värdet till ett signed heltal.
int alt_up_accelerometer_spi_read_z_axis (alt_up_accelerometer_spi_dev *accel_spi, alt_32 *z_axis);	Läser Z-axelvärdet från båda registren från accelerometer och omvandlar värdet till ett signed heltal.

7.3 C koden

I figur 11 visar c-koden för att validerar WATCHDOG_TIMER komponent på mjukvaran.

```
#include <system.h>
#include <alt_types.h>
#include <stdio.h>
#include <altera_avalon_mutex.h>
#include <draw_vga.h>
#include <math.h>
#include <skarm_funktioner.h> //enhet för att rita presentation- läg och löggnings- läge i skärmen.

int main(void) {
    alt_32 x_data=0, y_data=0, z_data=0;
    int i = 0, n_pos=0, b=0;

    // Pointer to shared variable
    int* shared_value = (int*)SHARED_MEMORY_BASE;

    // Declare pointer to mutex device and open the mutex.
    // This mutex is common to both cpu:s in the system.
    alt_mutex_dev* mutex = altera_avalon_mutex_open(MUTEX_0_NAME);
    printf("CPU extra (1) Start. ");

    clear_screen(BLACK);
    while(1) {
        // Try to aquire the mutex (successful if == 0)
        if(altera_avalon_mutex_trylock(mutex, 1) == 0)
        {
            /*CPU(2) sparade en värde i Shared memory från accelerometer*/
            if(*shared_value !=0){
                b++;
                if(b==1){
                    x_data = *shared_value;
                }else if(b==2){
                    y_data = *shared_value;
                }else{
                    z_data = *shared_value;
                    n_pos = 1;
                    b = 0;
                }
                /*initierar värde i shared memory för att CPU(2)
                 * kan läsa näsa verde från accelerometer*/
                *shared_value = 0;
            }
            // Unlock the mutex so it can be locked by other processors.
            altera_avalon_mutex_unlock(mutex);
        }
        for(i=0; i<500000; i++) {}

        if(n_pos == 1){
            printf("CPU (1).===>\n");
            printf("x_data : %d\n",x_data);
            printf("y_data : %d\n",y_data);
            printf("z_data : %d\n",z_data);

            clear_del_screen(5,10,320,240,BLACK);
            print_rektangel(x_data, y_data,20,GREEN);
            print_str(80,3,"X: ",YELLOW,0);
            print_str(140,3,"Y: ",RED,0);
            print_str(200,3,"Z: ",BLUE,0);
            int_print(92,3,x_data,3,YELLOW,0);
            int_print(152,3,y_data,3,RED,0);
            int_print(212,3,z_data,3,BLUE,0);
            x_data=0;
            y_data=0;
            z_data=0;
            n_pos=0;
        }
    }
    return 0;
}
```

Figur 9. MULTIPROCESSOR SYSTEM validering – CPU(1)

```c
#include <io.h>
#include <stdio.h>
#include <altera_avalon_mutex.h>
#include "altera_up_avalon_accelerometer_spi.h"
#include <altera_avalon_timer_regs.h>   //device driver for timer ip
#include <system.h>
#include <math.h>

void delay_hertz_second(int *param_hertz);

int main(void) {
    alt_32 x_data=0, y_data=0, z_data=0, xyz_value;
    int value_hertz = 5000000, n_pos;
    // Pointer to shared variable
    int* shared_value = (int*)SHARED_MEMORY_BASE;

    /* Initialize accelerometer */
    alt_up_accelerometer_spi_dev* accelerometer=alt_up_accelerometer_spi_open_dev(ACCELEROMETER_SPI_0_NAME);
    alt_u8 accelerometer_available=(accelerometer!=0);

    if(!accelerometer_available) {
        printf("Failed to obtain accelerometer device pointer\n");
    }

    // Declare pointer to mutex device and open the mutex.
    // This mutex is common to both cpus in the system.
    alt_mutex_dev* mutex = altera_avalon_mutex_open(MUTEX_0_NAME);

    printf("CPU (2) Start.\n");

    // cpu_0 will start by writing "0" the shared memory
    *shared_value = 0;
    while(1) {
    /* Read accelerometer values */
        alt_up_accelerometer_spi_read_x_axis(accelerometer,&x_data);
        alt_up_accelerometer_spi_read_y_axis(accelerometer,&y_data);
        alt_up_accelerometer_spi_read_z_axis(accelerometer,&z_data);

        n_pos = 0;
        //börjar från col=5 och fil=10
        if (x_data >= 5 && x_data <= 310){
            if (y_data >= 10 && y_data <= 215){
                n_pos = 1;
            }
        }
        /*om finns korrekt koordinater från accelerometer
         * börjar spara värde på shared memory*/
        if (n_pos == 1){
            printf("CPU (2).===>\n");
            for(alt_u32 b=1; b <= 3; b++)
            {
                /*separerar accelerometer-signalerna*/
                if(b==1){
                    xyz_value = x_data;
                    printf("x_data : %d\n",x_data);
                }else if(b==2){
                    xyz_value = y_data;
                    printf("y_data : %d\n",y_data);
                }else{
                    xyz_value = z_data;
                    printf("z_data : %d\n",z_data);
                }
                if(altera_avalon_mutex_trylock(mutex, 1) == 0) {
                    *shared_value = xyz_value;
                    // Unlock the mutex so it can be locked by other processors.
                    altera_avalon_mutex_unlock(mutex);
                }
                /*Väntar att CPU(1) läser värde från shared memory
                 * och initierar shared_value till noll för att fortsätta
                 * läsningen av nästa accelerometer värde; det är en
                 * kontroll inom CPU(2)*/
                while(*shared_value != 0) {delay_hertz_second(&value_hertz);}

            }
        }
    }
    return 0;
}

void delay_hertz_second(int *param_hertz) {
    TIMER_RESET();
    TIMER_START();
    while (TIMER_READ()<*param_hertz); //Timer räknar med 50MHz,  100000= 10hz det är max snabb med 10hz
}
```

Figur 10. MULTIPROCESSOR SYSTEM validering – CPU(2)